boba_extension_dim11

Dimension 11: Security Model & Privacy Architecture

Boba Torrent Browser Extension — Comprehensive Security Architecture

Date: 2025-06-25 **Scope**: Security and privacy architecture for a BitTorrent browser extension communicating with a self-hosted Boba service **Target**: Manifest V3 (MV3) Chrome/Firefox/Edge extensions

---

Table of Contents

---

1. Executive Summary

The Boba Torrent Browser Extension operates in a high-risk security environment: it interacts with potentially untrusted torrent sites, communicates with a user-self-hosted Boba service, handles magnet links and torrent metadata, and must protect user credentials and browsing data. This document provides a comprehensive security architecture based on Chrome Extension Manifest V3 security model best practices.

Key Design Principles

Principle	Implementation
Least Privilege	Request only minimum permissions; use optional permissions for advanced features
Defense in Depth	Multiple security layers: CSP, isolated worlds, HTTPS, input validation, encryption
Zero Trust	Validate all inputs, all senders, all origins; never hardcode credentials
Secure by Default	HTTPS-only communication, encrypted storage, strict CSP out of the box
Privacy First	Minimal data collection, local-only processing where possible, user opt-out

2. Threat Model (STRIDE Analysis)

2.1 STRIDE Classification for Boba Extension

Claim: STRIDE is a threat modeling framework created by Microsoft that classifies threats into six categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. Source: Security Compass URL: https://www.securitycompass.com/blog/stride-in-threat-modeling/ Date: 2025-08-25 Excerpt: “STRIDE is a threat modeling framework created by Microsoft that helps teams identify potential security threats by classifying them into six categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege” Context: Foundational threat modeling methodology applied to browser extension architecture Confidence: high

Spoofing (Authentication Violation)

Threat ID	Threat	Attack Vector	Mitigation
S1	Attacker spoofs Boba server	DNS hijacking, rogue access point	Certificate pinning/ validation, HTTPS-only
S2	Attacker spoofs extension popup	Clickjacking, fake extension page	`chrome-extension://` origin validation, UUID randomization
S3	Malicious website impersonates content script	XSS on torrent site	Isolated world execution, strict `matches` patterns
S4	Attacker spoofs magnet link	Fake torrent with attacker-controlled hash	Magnet link hash validation, info-hash format verification

Tampering (Integrity Violation)

Threat ID	Threat	Attack Vector	Mitigation
T1	Tampering with extension files	Local malware modifies extension code	Store-signed packages, integrity verification on install
T2	Tampering with Boba API requests	MITM intercepts extension↔Boba traffic	HTTPS-only, certificate validation, HSTS
T3	Tampering with stored credentials	Local malware extracts stored data	AES-GCM encryption of credentials in chrome.storage
T4	Tampering with torrent metadata	Rogue tracker returns fake peer data	Info-hash validation, peer ID verification

Repudiation (Non-Repudiation Violation)

Threat ID	Threat	Attack Vector	Mitigation
R1	User denies adding torrent	No audit trail	Local audit log of all Boba API operations (optional)
R2	Attacker covers tracks after exploiting extension	Insufficient logging	Structured logging (opt-in), tamper-resistant timestamps

Information Disclosure (Confidentiality Violation)

Threat ID	Threat	Attack Vector	Mitigation
I1	Boba credentials leaked	Unencrypted storage	AES-256-GCM encryption with PBKDF2 key derivation
I2	Browsing history leaked via extension	Overly broad host permissions	Minimal `matches` patterns, no `<all_urls>`
I3	Extension internal state exposed	`web_accessible_resources` misconfiguration	Restrict `web_accessible_resources`, use `use_dynamic_url`
I4	Torrent list/ metadata exposed	Insecure message passing	Sender validation (`sender.id`, `sender.url`) on all messages
I5	User's self-hosted Boba URL exposed	Leaked via analytics/ error reporting	No telemetry of server URLs, local-only error handling

Denial of Service (Availability Violation)

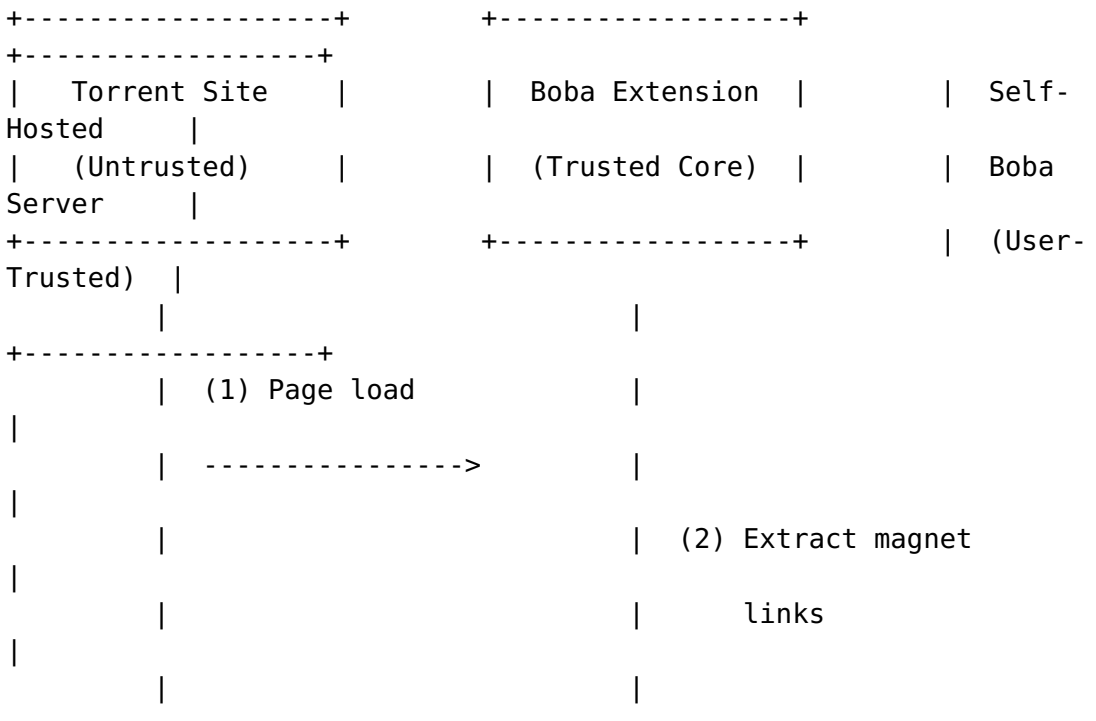
Threat ID	Threat	Attack Vector	Mitigation
D1	Boba server overwhelmed	Rapid-fire API requests	Client-side rate limiting with chrome.alarms
D2	Extension frozen by		`run_at: "document_idle",`

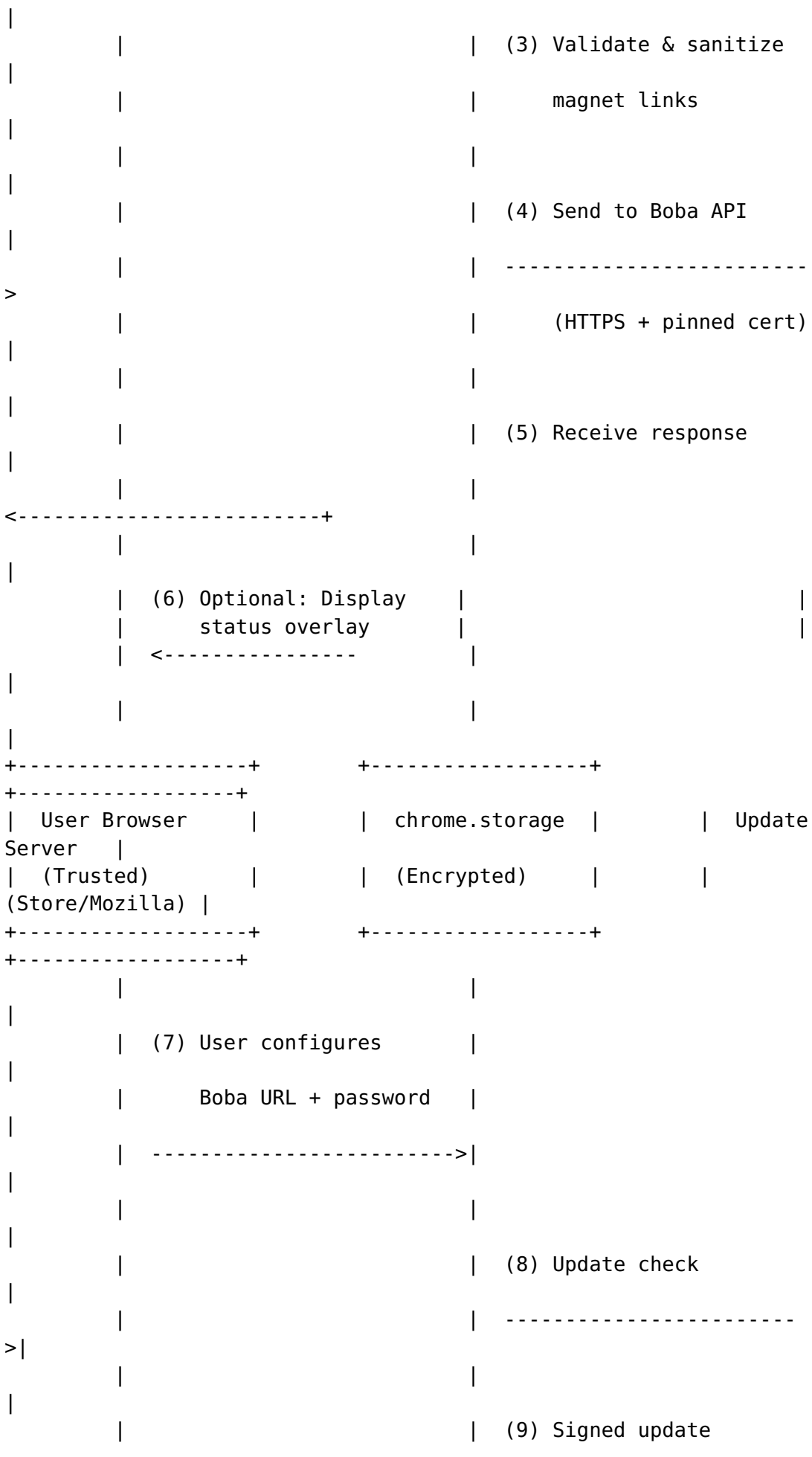
Threat ID	Threat	Attack Vector	Mitigation
	malicious page	Resource exhaustion in content script	sandboxed execution
D3	Browser storage exhausted	Malicious torrent metadata fills storage	Storage quotas, size limits on cached data

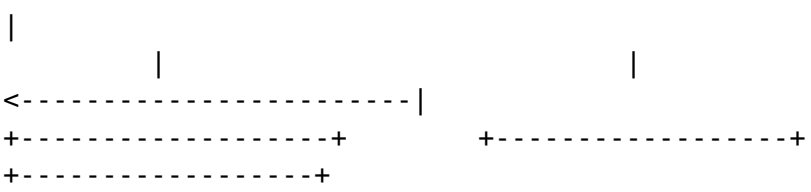
Elevation of Privilege (Authorization Violation)

Threat ID	Threat	Attack Vector	Mitigation
E1	Website gains extension API access	Compromised content script → background message	Strict sender validation, action allow-listing
E2	Content script escapes isolated world	V8 exploit, side-channel attack	Keep Chrome updated, minimal content script surface
E3	Arbitrary code execution via eval	Malicious input passed to background	No eval/new Function, strict CSP, input validation

2.2 Data Flow Diagram & Trust Boundaries




(3) Validate & sanitize
magnet links
(4) Send to Boba API
(HTTPS + pinned cert)
(5) Receive response
(6) Optional: Display
status overlay
User Browser
(Trusted)
chrome.storage
(Encrypted)
Update
Server
(Store/Mozilla)
(7) User configures
Boba URL + password
(8) Update check
(9) Signed update



Trust Boundaries: - **TB1**: Torrent site ↔ Content script (untrusted → semi-trusted) - **TB2**: Content script ↔ Background service worker (semi-trusted → trusted) - **TB3**: Extension ↔ Self-hosted Boba (trusted ↔ user-trusted) - **TB4**: Extension ↔ Update server (trusted ↔ vendor-trusted)

---

3. Manifest Permissions Architecture

3.1 Minimal Permission Manifest

Claim: MV3 disallows remotely hosted code and requires host permissions to be specified separately from API permissions. The `content_security_policy` in MV3 is an object with members for `extension_pages` and `sandbox` rather than a string. Source: CSS-Tricks / Chrome Developer Documentation URL: https://css-tricks.com/how-to-transition-to-manifest-v3-for-chrome-extensions/ Date: 2023-01-19 Excerpt: “In Manifest V3, host permissions are a separate element, and you should specify them in the `host_permissions` field… The content security policy in Manifest V3 is an object with different members representing alternative CSP contexts.” Context: Core MV3 security architecture changes Confidence: high

```
{
  "manifest_version": 3,
  "name": "Boba Torrent Manager",
  "version": "1.0.0",
  "description": "Send torrents and magnet links to your self-
       hosted Boba server",
  "permissions": [
    "storage",
    "alarms",
    "notifications",
    "activeTab"
  ],
  "optional_permissions": [
    "clipboardWrite"
  ],
  "host_permissions": [
    "http://localhost:*/",
    "https://localhost:*/"
  ],
  "optional_host_permissions": [
```

```
    "https://*/*"
  ],
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "action": {
    "default_popup": "popup.html",
    "default_icon": {
      "16": "icons/icon16.png",
      "32": "icons/icon32.png",
      "48": "icons/icon48.png",
      "128": "icons/icon128.png"
    }
  },
  "content_scripts": [
    {
      "matches": [
        "https://*.thepiratebay.*/*",
        "https://*.1337x.*/*",
        "https://*.nyaa.*/*",
        "https://*.rutracker.*/*"
      ],
      "js": ["content.js"],
      "run_at": "document_idle",
      "all_frames": false
    }
  ],
  "web_accessible_resources": [
    {
      "resources": ["icons/*.png"],
      "matches": ["https://*/*"],
      "use_dynamic_url": true
    }
  ],
  "content_security_policy": {
    "extension_pages": "default-src 'self'; script-src 'self';
        object-src 'self'; connect-src 'self' https:; img-src
        'self' data:; style-src 'self' 'unsafe-inline';",
    "sandbox": "sandbox allow-scripts; default-src 'self'; script-
        src 'self' 'unsafe-inline';"
  },
  "icons": {
    "16": "icons/icon16.png",
    "32": "icons/icon32.png",
    "48": "icons/icon48.png",
    "128": "icons/icon128.png
  }
}
```

3.2 Permission Justification Table

Permission	Required?	Justification	Risk if Absent
`storage`	**Yes**	Persist encrypted Boba URL, credentials, user preferences	User must re-enter credentials on every session
`alarms`	**Yes**	Rate limiting API calls to Boba server; periodic health checks	No rate limiting; Boba server could be overwhelmed
`notifications`	**Yes**	Notify user of successful/ failed torrent additions	Silent failures; user uncertainty
`activeTab`	**Yes**	Read current tab URL for context menu magnet link detection	Cannot detect magnet links on current page
`clipboardWrite`	Optional	Copy magnet links to clipboard on user request	Feature unavailable
`host_permissions` (localhost)	**Yes**	Connect to self-hosted Boba server on localhost	Cannot communicate with Boba
`optional_host_permissions` (https)	Optional	Connect to Boba hosted on non-localhost domain	Boba must be on localhost only

3.3 Why NOT Request These Permissions

Permission	Why Excluded	Risk if Included
`<all_urls>`	Far too broad; allows access to all websites	Any website can exploit content script vulnerabilities

Permission	Why Excluded	Risk if Included
`tabs`	Not needed if using `activeTab` pattern	Can read all tab URLs, enabling browsing history tracking
`cookies`	Boba uses its own authentication, not browser cookies	Can exfiltrate session cookies from any website
`webRequest`	MV3 restricts this; use declarativeNetRequest instead	Can intercept and modify all network traffic
`scripting`	Not needed if content scripts declared in manifest	Can execute arbitrary code on any page
`downloads`	Torrents are handled by Boba, not the extension	Can monitor and manipulate all browser downloads
`history`	No need to access browser history	Can exfiltrate complete browsing history

4. Host Permissions & Pattern Matching

4.1 Torrent Site Match Patterns

Claim: Content scripts operate in an isolated world and cannot access JavaScript variables on the web page. However, they share access to the page's DOM and can receive `postMessage` messages. The protection offered by event source checks is nullified if the content script uses wildcard `matches` patterns. Source: Space Raccoon Security Research URL: https://spaceraccoon.dev/universal-code-execution-browser-extensions/ Date: 2024-07-07 Excerpt: "Notably, the protection offered by the event source check is completely nullified if the content script is injected with a wildcard `matches` pattern, since this means any origin can still trigger this web page-content script-background script channel simply by sending a postMessage to itself." Context: Critical security consideration for content script match patterns Confidence: high

```
{
  "content_scripts": [
    {
      "matches": [
        "https://*.thepiratebay.*/*",
```

```
        "https://*.1337x.*/*",
        "https://*.nyaa.*/*",
        "https://*.rutracker.*/*",
        "https://*.yts.*/*",
        "https://*.eztv.*/*",
        "https://*.torrentgalaxy.*/*",
        "https://*.limetorrents.*/*",
        "https://*.torlock.*/*",
        "https://*. zooqle.*/*"
      ],
      "js": ["content.js"],
      "run_at": "document_idle",
      "all_frames": false
    }
  ]
}
```

4.2 Host Permission Patterns for Boba Server

The self-hosted Boba server URL is user-configurable. The extension dynamically requests host permissions using the `permissions` API:

```
// Background service worker: Request permission for Boba server
async function requestBobaHostPermission(bobaUrl) {
  try {
    const url = new URL(bobaUrl);
    const pattern = `${url.protocol}//${url.hostname}${url.port ?
        ':' + url.port : ''}/*`;

    const granted = await chrome.permissions.request({
      origins: [pattern]
    });

    return granted;
  } catch (error) {
    console.error('Invalid Boba URL:', error);
    return false;
  }
}
```

4.3 Wildcard Considerations

Pattern Type	Example	Risk Level	Recommendation
Specific domain	`https://boba.mylan/*`	Low	**Preferred** for known Boba servers
Localhost with port		Low	**Acceptable** for local development

Pattern Type	Example	Risk Level	Recommendation
	`http://localhost:9090/*`		
Localhost wildcard	`http://localhost:*/`	Medium	Acceptable; restricts to localhost
Domain wildcard	`https://*.example.com/*`	Medium	Use only if Boba on subdomain
`<all_urls>`	N/A	**Critical**	**Never use**

5. Credential Storage & Encryption

5.1 Storage Architecture

Claim: Chrome's `chrome.storage.local` is not encrypted by default — "Confidential user information should not be stored! The storage area isn't encrypted." Client-side encryption using Web Crypto API with AES-GCM is the recommended approach. Source: Stack Overflow / Chrome Extension Documentation URL: https://stackoverflow.com/questions/20130126/security-concerns-for-using-localstorage-or-chrome-storage-inside-chrome-extensi Date: 2013-11-22 (still applicable; MV3 storage remains unencrypted) Excerpt: "The storage area isn't encrypted… You could use some sort of a library to encrypt local data and make users enter a passphrase to decrypt the data." Context: chrome.storage.local stores data in plaintext; encryption must be implemented by the extension Confidence: high

Claim: `chrome.storage.session` (MV3, Chrome 102+) stores data in memory only and does not persist to disk. This is ideal for temporary sensitive data like decrypted session tokens. Source: Chrome Developer Documentation URL: https://developer.chrome.com/docs/extensions/reference/api/storage Date: 2026-05-12 Excerpt: "`session`: Chrome 102+, MV3+. Items in the `session` storage area are stored in memory and are not persisted to disk." Context: Session storage provides volatile, memory-only storage for sensitive transient data Confidence: high

5.2 Encryption Implementation (Web Crypto API)

Claim: The Web Crypto API with AES-GCM and PBKDF2 key derivation provides a secure method for encrypting sensitive data in browser extensions without sending it to a server. Source: Medium / CodeGuyAkash URL: https://codeguyakash.medium.com/securely-encrypting-data-in-a-chrome-

extension-without-sending-it-to-a-server-key-management-e53b78fd865b Date: 2026-03-04 Excerpt: “For encryption I am using the Web Crypto API with AES-GCM, and deriving the key from a user password using PBKDF2.” Context: Practical implementation of client-side credential encryption Confidence: high

```
// crypto-utils.js — Credential encryption module

const ALGO = 'AES-GCM';
const KEY_LENGTH = 256;
const IV_LENGTH = 12;
const SALT_LENGTH = 16;
const PBKDF2_ITERATIONS = 100000;

/**
 * Derive an AES-GCM key from a password using PBKDF2
 * @param {string} password - User's master password
 * @param {Uint8Array} salt - Random salt
 * @returns {Promise<CryptoKey>} Derived AES-GCM key
 */
export async function deriveKey(password, salt) {
  const enc = new TextEncoder();
  const keyMaterial = await crypto.subtle.importKey(
    'raw',
    enc.encode(password),
    'PBKDF2',
    false,
    ['deriveKey']
  );
  return crypto.subtle.deriveKey(
    {
      name: 'PBKDF2',
      salt,
      iterations: PBKDF2_ITERATIONS,
      hash: 'SHA-256'
    },
    keyMaterial,
    { name: ALGO, length: KEY_LENGTH },
    false,  // Not extractable
    ['encrypt', 'decrypt']
  );
}

/**
 * Encrypt sensitive data with AES-GCM
 * @param {Object} data - Data to encrypt (must be JSON-
       serializable)
 * @param {string} password - User's master password
 * @returns {Promise<{cipher: string, iv: string, salt: string}>}
 */
export async function encryptCredentials(data, password) {
```

```
  const enc = new TextEncoder();
  const iv = crypto.getRandomValues(new Uint8Array(IV_LENGTH));
  const salt = crypto.getRandomValues(new
        Uint8Array(SALT_LENGTH));
  const key = await deriveKey(password, salt);

  const encryptedBuffer = await crypto.subtle.encrypt(
    { name: ALGO, iv },
    key,
    enc.encode(JSON.stringify(data))
  );

  return {
    cipher: bufferToBase64(encryptedBuffer),
    iv: bufferToBase64(iv),
    salt: bufferToBase64(salt)
  };
}

/**
 * Decrypt sensitive data
 * @param {Object} encryptedPayload - Object with cipher, iv, salt
 * @param {string} password - User's master password
 * @returns {Promise<Object>} Decrypted data
 */
export async function decryptCredentials(encryptedPayload,
        password) {
  const { cipher, iv, salt } = encryptedPayload;
  const key = await deriveKey(password, base64ToBuffer(salt));

  const decryptedBuffer = await crypto.subtle.decrypt(
    { name: ALGO, iv: base64ToBuffer(iv) },
    key,
    base64ToBuffer(cipher)
  );

  const dec = new TextDecoder();
  return JSON.parse(dec.decode(decryptedBuffer));
}

// Helper functions
function bufferToBase64(buffer) {
  const bytes = new Uint8Array(buffer);
  let binary = '';
  for (let i = 0; i < bytes.byteLength; i++) {
    binary += String.fromCharCode(bytes[i]);
  }
  return btoa(binary);
}

function base64ToBuffer(base64) {
  const binary = atob(base64);
```

```
  const bytes = new Uint8Array(binary.length);
  for (let i = 0; i < binary.length; i++) {
    bytes[i] = binary.charCodeAt(i);
  }
  return bytes;
}
```

5.3 Storage Layer

```
// storage.js — Secure storage operations

const STORAGE_KEY_CREDENTIALS = 'boba_credentials_encrypted';
const STORAGE_KEY_SETTINGS = 'boba_settings';

/**
 * Store encrypted Boba credentials
 * @param {Object} credentials - { url, username, password,
       apiKey }
 * @param {string} masterPassword - User-provided master password
 */
export async function storeCredentials(credentials,
       masterPassword) {
  const encrypted = await encryptCredentials(credentials,
       masterPassword);
  await chrome.storage.local.set({
    [STORAGE_KEY_CREDENTIALS]: encrypted
  });
}

/**
 * Retrieve and decrypt Boba credentials
 * @param {string} masterPassword - User-provided master password
 * @returns {Promise<Object|null>} Decrypted credentials or null
 */
export async function getCredentials(masterPassword) {
  try {
    const result = await
        chrome.storage.local.get(STORAGE_KEY_CREDENTIALS);
    if (!result[STORAGE_KEY_CREDENTIALS]) return null;

    return await decryptCredentials(
      result[STORAGE_KEY_CREDENTIALS],
      masterPassword
    );
  } catch (error) {
    // Decryption failed — wrong password or corrupted data
    console.error('Failed to decrypt credentials:', error);
    return null;
  }
}

/**
```

```
 * Store non-sensitive settings (unencrypted)
 * @param {Object} settings
 */
export async function storeSettings(settings) {
  await chrome.storage.local.set({
    [STORAGE_KEY_SETTINGS]: settings
  });
}

/**
 * Cache decrypted credentials in session storage (volatile,
       memory-only)
 * for the current browser session. Lost on browser restart.
 * @param {Object} decryptedCredentials
 */
export async function
       cacheCredentialsInSession(decryptedCredentials) {
  await chrome.storage.session.set({
    boba_session_cache: decryptedCredentials
  });
}

/**
 * Get cached credentials from session storage
 * @returns {Promise<Object|null>}
 */
export async function getCachedCredentials() {
  const result = await
       chrome.storage.session.get('boba_session_cache');
  return result.boba_session_cache || null;
}

/**
 * Clear all stored credentials (logout)
 */
export async function clearAllCredentials() {
  await chrome.storage.local.remove(STORAGE_KEY_CREDENTIALS);
  await chrome.storage.session.remove('boba_session_cache');
}
```

5.4 Security Properties

Property	Implementation
Algorithm	AES-256-GCM (authenticated encryption)
Key Derivation	PBKDF2-HMAC-SHA256, 100,000 iterations
Salt	128-bit random per encryption
IV	96-bit random per encryption (GCM recommended)
Key Storage	Non-extractable CryptoKey, not persisted

Property	Implementation
Master Password	Never stored; provided by user on each session
Plaintext Storage	Only encrypted data hits disk

6. Content Script Isolation

6.1 Isolated World Architecture

Claim: Content scripts run in an “isolated world” — they cannot access JavaScript variables or functions defined by web pages or by other content scripts. They share only the DOM with the page. This prevents page JavaScript from accessing extension APIs. Source: Chrome Developer Documentation / OWASP URL: https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts Date: 2012-09-17 (updated) Excerpt: “Content scripts execute in a special environment called an isolated world. They have access to the DOM of the page they are injected into, but not to any JavaScript variables or functions created by the page.” Context: Core security model of content scripts Confidence: high

6.2 Content Script Security Patterns

```
// content.js — Secure content script implementation

// SECURE: Use JSON.parse instead of eval for parsing data
function safelyParseJSON(data) {
  try {
    return JSON.parse(data);  // Safe — doesn't execute code
  } catch (e) {
    return null;
  }
}

// SECURE: Use textContent instead of innerHTML to prevent XSS
function safelyDisplayStatus(element, message) {
  element.textContent = message;  // Safe — escapes HTML
}

// SECURE: Validate all magnet links before sending to background
function isValidMagnetLink(href) {
  if (!href || typeof href !== 'string') return false;

  // Must start with magnet:?xt=urn:btih: or magnet:?xt=urn:btmh:
  const magnetRegex = /^magnet:\?xt=urn:bti[hm]:[a-fA-F0-9]
        {40,64}/;
```

```
  return magnetRegex.test(href);
}

// SECURE: Extract magnet links from the page
function extractMagnetLinks() {
  const links = document.querySelectorAll('a[href^="magnet:"]');
  const magnets = [];

  for (const link of links) {
    const href = link.getAttribute('href');
    if (isValidMagnetLink(href)) {
      magnets.push({
        url: href,
        name: link.textContent.trim() || 'Unknown',
        timestamp: Date.now()
      });
    }
  }

  return magnets;
}

// SECURE: Message passing to background with origin validation
// NEVER trust responses from the page without validation
function sendToBackground(message) {
  // Only send validated, structured data
  chrome.runtime.sendMessage({
    action: 'VALID_ACTION_NAME',  // Action allow-listing
    data: message
  });
}

// SECURE: Handle messages from background
chrome.runtime.onMessage.addListener((request, sender,
        sendResponse) => {
  // Validate sender is our extension
  if (sender.id !== chrome.runtime.id) return;

  switch (request.action) {
    case 'GET_MAGNETS':
      sendResponse({ magnets: extractMagnetLinks() });
      break;
    case 'SHOW_STATUS':
      safelyDisplayStatus(document.getElementById('boba-status'),
        request.message);
      break;
  }

  return true;
});
```

6.3 Content Script Anti-Patterns to Avoid

Anti-Pattern	Risk	Secure Alternative
`element.innerHTML = userInput`	XSS	`element.textContent = userInput`
`eval(jsonData)`	RCE	`JSON.parse(jsonData)`
`setTimeout(stringCode, delay)`	RCE	`setTimeout(() => functionCall(), delay)`
`matches: ["<all_urls>"]`	Universal attack surface	Specific torrent site patterns
`all_frames: true`	iframe exploitation	`all_frames: false` unless required
`window.addEventListener('message', ...)` without origin check	postMessage hijacking	Always validate `event.origin`

---

7. Content Security Policy (CSP)

7.1 Default CSP for Extension Pages

Claim: In MV3, the CSP is an object with members for `extension_pages` and `sandbox`. The default policy is `script-src 'self'; object-src 'self';`. Manifest V3 does not allow values that permit remote code execution in `extension_pages` CSP — `script-src`, `object-src`, and `worker-src` directives can only be `'self'`, `'none'`, or `'wasm-unsafe-eval'`. Source: Chrome Developer Documentation URL: https://developer.chrome.com/docs/extensions/reference/manifest/content-security-policy Date: 2024-02-13 Excerpt: “The ‘extension page’ policy applies to extension’s web page and worker contexts. This includes the extension popup, background worker, and tabs opened by the extension… If a user does not define a CSP in the manifest, the extension pages and sandboxed extension pages will use default properties.” Context: Official MV3 CSP configuration reference Confidence: high

Claim: MV3 CSP `extension_pages` does not allow `unsafe-inline`, `unsafe-eval`, or remote script sources. These restrictions prevent inline scripts and remote code execution. Source: Chrome Developer Documentation — Improve Extension Security URL: https://developer.chrome.com/docs/extensions/develop/migrate/improve-security Date: 2023-03-08 Excerpt: “Manifest V3 does not allow certain content security policy values in the `extension_pages`

field that were allowed in Manifest V2. Specifically, Manifest V3 does not allow values that permit remote code execution.”
Context: CSP restrictions for MV3 extensions Confidence: high

```
{
  "content_security_policy": {
    "extension_pages": "default-src 'self'; script-src 'self';
        object-src 'self'; connect-src 'self' https:; img-src
        'self' data:; style-src 'self' 'unsafe-inline';",
    "sandbox": "sandbox allow-scripts allow-forms allow-popups
        allow-modals; script-src 'self' 'unsafe-inline' 'unsafe-
        eval'; child-src 'self';"
  }
}
```

7.2 CSP Directive Breakdown

Directive	Value	Purpose
`default-src`	`'self'`	Default fallback: only allow same-origin resources
`script-src`	`'self'`	Only execute scripts bundled with the extension
`object-src`	`'self'`	Only allow extension-bundled plugins/objects
`connect-src`	`'self' https:`	Allow fetch/XHR to extension origin and any HTTPS
`img-src`	`'self' data:`	Allow extension images and data URIs
`style-src`	`'self' 'unsafe-inline'`	Allow extension styles; `'unsafe-inline'` required for some UI frameworks
`worker-src`	`'self'` (default)	Only allow extension service workers

7.3 CSP for Different Contexts

```
// CSP applied to different extension contexts

// popup.html — Uses 'extension_pages' CSP
// options.html — Uses 'extension_pages' CSP
```

```
// background.js (service worker) — Uses 'extension_pages' CSP
// sandboxed.html — Uses 'sandbox' CSP (more permissive)

// Inline scripts are BLOCKED in extension pages.
// Must use external script files:

//    CORRECT: External script
// popup.html
<script src="popup.js"></script>

//    WRONG: Inline script (violates CSP)
// popup.html
<script>
  console.log('This will be blocked!');
</script>
```

7.4 Handling CSP Violations

```
// Report CSP violations for debugging
// Note: MV3 extensions cannot use CSP report-uri for external
        URLs
// Log violations locally for debugging

chrome.webRequest?.onErrorOccurred?.addListener(
  (details) => {
    if (details.error.includes('Content Security Policy')) {
      console.warn('CSP violation detected:', details);
    }
  },
  { urls: ['chrome-extension://*/*'] }
);
```

---

8. HTTPS Enforcement

8.1 HTTPS-Only Communication

```
// background.js — HTTPS enforcement for Boba API

/**
 * Validate that a URL uses HTTPS (except localhost)
 * @param {string} url - URL to validate
 * @returns {boolean}
 */
function isSecureUrl(url) {
  try {
    const parsed = new URL(url);

    // Allow HTTP for localhost/127.0.0.1 only
```

```
    if (parsed.hostname === 'localhost' || parsed.hostname ===
        '127.0.0.1') {
      return true;
    }

    // Require HTTPS for all other hosts
    return parsed.protocol === 'https:';
  } catch {
    return false;
  }
}

/**
 * Secure fetch wrapper for Boba API calls
 * @param {string} endpoint - Boba API endpoint
 * @param {Object} options - fetch options
 * @param {Object} credentials - Decrypted credentials
 */
async function bobaFetch(endpoint, options = {}, credentials) {
  const bobaUrl = credentials.url;

  // Enforce HTTPS
  if (!isSecureUrl(bobaUrl)) {
    throw new SecurityError(
      'Boba server URL must use HTTPS (except localhost). ' +
      'Non-HTTPS connections are blocked for security.'
    );
  }

  const url = new URL(endpoint, bobaUrl).toString();

  // Build headers with authentication
  const headers = new Headers({
    'Content-Type': 'application/json',
    'X-Requested-With': 'XMLHttpRequest',
    ...options.headers
  });

  // Add API key if available
  if (credentials.apiKey) {
    headers.set('X-API-Key', credentials.apiKey);
  }

  // Add Basic Auth if username/password provided
  if (credentials.username && credentials.password) {
    const basicAuth = btoa(`${credentials.username}:$
        {credentials.password}`);
    headers.set('Authorization', `Basic ${basicAuth}`);
  }

  return fetch(url, {
    ...options,
```

```
    headers,
    // Security options
    credentials: 'omit',           // Don't send browser cookies
    referrerPolicy: 'no-referrer', // Don't leak referrer
    cache: 'no-store'              // Don't cache sensitive
        responses
  });
}

class SecurityError extends Error {
  constructor(message) {
    super(message);
    this.name = 'SecurityError';
  }
}
```

8.2 HTTP Strict Transport Security (HSTS)

The extension should validate that the Boba server sends HSTS headers:

```
/**
 * Check if Boba server supports HSTS
 * @param {Response} response - fetch Response object
 */
function validateHSTS(response) {
  const hstsHeader = response.headers.get('strict-transport-
        security');

  if (!hstsHeader && !isLocalhost(response.url)) {
    console.warn('Boba server does not send HSTS header. ' +
      'Consider enabling HSTS on your server for improved
        security.');
    // Non-blocking: warn but don't fail (self-hosted servers may
        not have HSTS)
  }

  return hstsHeader;
}
```

---

9. Certificate Validation for Self-Hosted Boba

9.1 Certificate Pinning Approach

Claim: For self-hosted servers with self-signed certificates, the recommended approach is certificate bundling and pinning — bundle the server's certificate in the extension and validate the runtime certificate against the bundled one. Alternatively, instruct

users to install the self-signed CA into their system's trusted root store. Source: BrowserStack URL: https://www.browserstack.com/blog/building-secure-native-apps-with-self-signed-ssl-certificates-using-certificate-pinning/ Date: 2021-05-07 Excerpt: "App developers can bundle or import the backend server's custom SSL certificate within the app's code repository. Once this is done, additional logic can be configured in the codebase to validate the actual self-signed certificate at runtime with the existing bundled certificate in the app." Context: Certificate pinning approach for self-signed certificates Confidence: medium (for browser extensions specifically)

9.2 Self-Signed Certificate Handling

```
// certificate-validator.js — Certificate validation for self-
        hosted Boba

/**
 * For self-hosted Boba servers with self-signed certificates,
 * users must install the certificate in their OS trust store.
 * The extension CANNOT programmatically trust self-signed certs
 * (browser security restriction).
 *
 * This module provides guidance and detection.
 */

/**
 * Check if the Boba server certificate is trusted.
 * fetch() will throw if the cert is not trusted.
 */
async function validateBobaCertificate(bobaUrl) {
  try {
    const response = await fetch(bobaUrl, {
      method: 'HEAD',
      mode: 'no-cors',
      cache: 'no-store'
    });

    // If fetch succeeds without error, certificate is trusted
    return { valid: true, error: null };
  } catch (error) {
    if (error.message.includes('CERT') ||
        error.message.includes('certificate') ||
        error.message.includes('SSL') ||
        error.message.includes('TLS')) {
      return {
        valid: false,
        error: 'CERTIFICATE_UNTRUSTED',
        message: `The Boba server's certificate is not trusted. `
        +
          `If using a self-signed certificate, please install it
        ` +
```

```
          `in your system's certificate trust store.`,
        details: error.message
      };
    }

    // Other error (network, timeout, etc.)
    return { valid: false, error: 'CONNECTION_FAILED', details:
        error.message };
  }
}

/**
 * Detect if URL uses a self-signed or custom CA certificate
 */
async function detectCertificateIssue(bobaUrl) {
  const result = await validateBobaCertificate(bobaUrl);

  if (!result.valid && result.error === 'CERTIFICATE_UNTRUSTED') {
    // Provide helpful instructions based on OS
    const instructions = getCertInstallInstructions();

    // Show notification to user
    chrome.notifications.create('cert-error', {
      type: 'basic',
      iconUrl: 'icons/icon48.png',
      title: 'Boba: Certificate Issue',
      message: `Cannot connect to Boba server due to certificate
        issue. Click for instructions.`,
      requireInteraction: true
    });

    return { ...result, instructions };
  }

  return result;
}

function getCertInstallInstructions() {
  const userAgent = navigator.userAgent;

  if (userAgent.includes('Mac OS X')) {
    return `To trust a self-signed certificate on macOS:
1. Double-click the .cer file
2. Select "System" keychain
3. Double-click the imported certificate
4. Expand "Trust" and select "Always Trust"`;
  }

  if (userAgent.includes('Windows')) {
    return `To trust a self-signed certificate on Windows:
1. Double-click the .cer file
2. Click "Install Certificate"
```

```
3. Select "Local Machine" → "Place all certificates in the
       following store"
4. Select "Trusted Root Certification Authorities"`;
  }

  if (userAgent.includes('Linux')) {
    return `To trust a self-signed certificate on Linux:
1. Copy the .crt file to /usr/local/share/ca-certificates/
2. Run: sudo update-ca-certificates
3. Restart Chrome`;
  }

  return 'Please install the self-signed certificate in your
       system trust store.';
}
```

9.3 Certificate Fingerprint Verification (Optional Advanced)

```
/**
 * Allow users to pin a specific certificate fingerprint.
 * This provides TOFU (Trust On First Use) security model.
 */

const SPKI_HEADER = new Uint8Array([0x30, 0x59, 0x30, 0x13, 0x06,
       0x07, 0x2a, 0x86, 0x48, 0xce, 0x3d, 0x02, 0x01, 0x06,
       0x08, 0x2a, 0x86, 0x48, 0xce, 0x3d, 0x03, 0x01, 0x07,
       0x03, 0x42, 0x00]);

/**
 * Extract SPKI fingerprint from a certificate
 * Note: This requires the Web Crypto API and certificate access
 * which is limited in extensions. This is a conceptual
       implementation.
 */
async function getCertificateFingerprint(certPem) {
  // Remove PEM headers
  const base64 = certPem
    .replace('-----BEGIN CERTIFICATE-----', '')
    .replace('-----END CERTIFICATE-----', '')
    .replace(/\s/g, '');

  const certBuffer = base64ToBuffer(base64);

  // Import certificate
  const cert = await crypto.subtle.importKey(
    'spki',
    certBuffer,
    { name: 'ECDSA', namedCurve: 'P-256' },
    false,
    []
  );
```

```
  // Export raw SPKI and hash
  const spki = await crypto.subtle.exportKey('spki', cert);
  const hash = await crypto.subtle.digest('SHA-256', spki);

  return bufferToHex(new Uint8Array(hash));
}

function bufferToHex(buffer) {
  return Array.from(buffer)
    .map(b => b.toString(16).padStart(2, '0'))
    .join('');
}
```

---

10. Input Validation

10.1 Magnet Link Validation

Claim: BitTorrent magnet links follow specific formats: v1 uses `xt=urn:btih:` with a 40-character hex-encoded SHA-1 info-hash; v2 uses `xt=urn:btmh:` with a SHA-256 hash. The info-hash hex encoded is 40 characters, and base32 encoded is 32 characters. Source: Wikipedia / BitTorrent BEP URL: https://en.wikipedia.org/wiki/Magnet_URI_scheme Date: 2005-01-24 (continuously updated) Excerpt: “BitTorrent info hash (BTIH): These are hex-encoded SHA-1 hash sums of the ‘info’ sections of BitTorrent metafiles… For backwards compatibility with existing links, clients should also support the Base32 encoded version of the hash.” Context: Magnet URI format specification Confidence: high

```
// validators.js — Input validation for torrent-related data

/**
 * Validate a magnet link URL
 * @param {string} url - URL to validate
 * @returns {{valid: boolean, infoHash: string|null, version:
       number, error: string|null}}
 */
export function validateMagnetLink(url) {
  if (!url || typeof url !== 'string') {
    return { valid: false, infoHash: null, version: null, error:
        'URL is required' };
  }

  // Must start with magnet:
  if (!url.startsWith('magnet:')) {
    return { valid: false, infoHash: null, version: null, error:
        'Not a magnet link' };
  }
```

```
  try {
    const params = new
        URLSearchParams(url.substring(8)); // After 'magnet:?'

    // Check for exact topic (xt) parameter
    const xt = params.get('xt');
    if (!xt) {
      return { valid: false, infoHash: null, version: null,
        error: 'Missing xt parameter' };
    }

    // Parse URN
    const btihMatch = xt.match(/^urn:btih:([a-fA-F0-9]{40}|[A-
        Z2-7]{32})$/i);
    const btmhMatch = xt.match(/^urn:btmh:1220([a-fA-F0-9]{64})$/
        i);

    if (btihMatch) {
      return {
        valid: true,
        infoHash: btihMatch[1].toLowerCase(),
        version: 1,
        error: null,
        displayName: params.get('dn') || 'Unknown',
        trackers: params.getAll('tr')
      };
    }

    if (btmhMatch) {
      return {
        valid: true,
        infoHash: btmhMatch[1].toLowerCase(),
        version: 2,
        error: null,
        displayName: params.get('dn') || 'Unknown',
        trackers: params.getAll('tr')
      };
    }

    return { valid: false, infoHash: null, version: null, error:
        'Invalid info-hash format' };
  } catch (error) {
    return { valid: false, infoHash: null, version: null, error:
        'Invalid magnet link format' };
  }
}

/**
 * Validate a .torrent file URL
 * @param {string} url - URL to validate
 * @returns {boolean}
 */
```

```javascript
export function validateTorrentUrl(url) {
  if (!url || typeof url !== 'string') return false;

  try {
    const parsed = new URL(url);

    // Must be HTTP or HTTPS
    if (parsed.protocol !== 'http:' && parsed.protocol !==
        'https:') {
      return false;
    }

    // Must end with .torrent
    if (!parsed.pathname.toLowerCase().endsWith('.torrent')) {
      return false;
    }

    // Reject URLs with credentials
    if (parsed.username || parsed.password) {
      return false;
    }

    // Reject URLs with fragments (not meaningful for torrents)
    if (parsed.hash) {
      return false;
    }

    // Block known malicious patterns
    const blockedPatterns = [
      /\.exe$/i,
      /\.dll$/i,
      /\.bat$/i,
      /\.sh$/i,
      /<script/i,
      /javascript:/i,
      /data:/i,
      /vbscript:/i
    ];

    for (const pattern of blockedPatterns) {
      if (pattern.test(url)) return false;
    }

    return true;
  } catch {
    return false;
  }
}

/**
 * Validate Boba server URL configuration
 * @param {string} url - Boba server URL
```

```
 * @returns {{valid: boolean, error: string|null, normalized:
       string|null}}
 */
export function validateBobaUrl(url) {
  if (!url || typeof url !== 'string') {
    return { valid: false, error: 'URL is required', normalized:
        null };
  }

  try {
    let normalized = url.trim();

    // Ensure protocol is present
    if (!normalized.startsWith('http://') && !
        normalized.startsWith('https://')) {
      normalized = 'https://' + normalized;
    }

    const parsed = new URL(normalized);

    // Reject URLs with credentials
    if (parsed.username || parsed.password) {
      return { valid: false, error: 'URL must not contain
        credentials', normalized: null };
    }

    // Reject file://, ftp://, etc.
    if (parsed.protocol !== 'http:' && parsed.protocol !==
        'https:') {
      return { valid: false, error:
        'Only HTTP and HTTPS protocols are supported',
        normalized: null };
    }

    // Require a hostname (not just IP patterns that could be
        internal)
    if (!parsed.hostname) {
      return { valid: false, error: 'Invalid hostname',
        normalized: null };
    }

    // Block internal/private IP ranges for non-localhost
        (security measure)
    if (parsed.hostname !== 'localhost' && parsed.hostname !==
        '127.0.0.1') {
      // Check for private IP ranges
      const privateIpRegex = /^(10\.|172\.(1[6-9]|2[0-9]|3[01])\.|
        192\.168\.|169\.254\.|127\.|0\.0\.0\.0)/;
      if (privateIpRegex.test(parsed.hostname)) {
        // Allow but warn — this is a self-hosted server after all
        console.warn('Boba server is on a private IP range:',
        parsed.hostname);
      }
    }
```

```
    // Normalize: remove trailing slash
    normalized = normalized.replace(/\/$/, '');

    return { valid: true, error: null, normalized };
  } catch (error) {
    return { valid: false, error: 'Invalid URL format: ' +
        error.message, normalized: null };
  }
}

/**
 * Sanitize a display name from a torrent/magnet link
 * @param {string} name - Raw display name
 * @returns {string} Sanitized name
 */
export function sanitizeDisplayName(name) {
  if (!name || typeof name !== 'string') return 'Unknown';

  return name
    .trim()
    .replace(/[<>\"']/g, '')  // Remove dangerous characters
    .substring(0, 255);          // Limit length
}

/**
 * Validate action name in messages (prevent action injection)
 * @param {string} action - Requested action
 * @returns {boolean}
 */
export function isValidAction(action) {
  const ALLOWED_ACTIONS = [
    'ADD_MAGNET',
    'ADD_TORRENT',
    'GET_STATUS',
    'GET_TORRENTS',
    'REMOVE_TORRENT',
    'GET_SETTINGS',
    'SAVE_SETTINGS',
    'TEST_CONNECTION',
    'VALIDATE_CREDENTIALS',
    'CLEAR_CREDENTIALS',
    'GET_MAGNETS'  // From content script
  ];

  return ALLOWED_ACTIONS.includes(action);
}
```

10.2 URL Sanitization

```
/**
 * Comprehensive URL sanitizer
 * Prevents SSRF, open redirect, and protocol injection
 */
export function sanitizeUrl(input) {
  if (!input || typeof input !== 'string') return null;

  try {
    const url = new URL(input);

    // Protocol allow-list
    const allowedProtocols = ['http:', 'https:', 'magnet:'];
    if (!allowedProtocols.includes(url.protocol)) {
      return null;
    }

    // Block URLs with embedded credentials
    url.username = '';
    url.password = '';

    // Block dangerous ports
    const dangerousPorts = [22, 23, 25, 53, 110, 135, 139, 143,
        445, 3306, 3389, 5432, 6379, 27017];
    if (url.port && dangerousPorts.includes(parseInt(url.port))) {
      return null;
    }

    return url.toString();
  } catch {
    return null;
  }
}
```

---

11. Rate Limiting

11.1 Client-Side Rate Limiting with chrome.alarms

Claim: The `chrome.alarms` API is the MV3-recommended approach for periodic tasks, replacing `setTimeout`/`setInterval` which are terminated with the service worker. Alarms survive service worker termination and persist across browser restarts. Source: Chrome Extension Guide / Chrome Developer Documentation URL: https://github.com/theluckystrike/chrome-extension-guide/blob/main/docs/mv3/service-workers.md Date: 2026-01-15 Excerpt: “Chrome provides the `chrome.alarms` API specifically for this purpose… Benefits of chrome.alarms: Survives service worker termination,

Alarms persist across browser restarts” Context: MV3 service worker rate limiting and periodic task implementation Confidence: high

```
// rate-limiter.js — Client-side rate limiting for Boba API

const RATE_LIMIT_PREFIX = 'rl_';
const DEFAULT_MAX_REQUESTS = 30;  // per minute
const DEFAULT_WINDOW_MS = 60000;  // 1 minute window

/**
 * Initialize rate limiter with user-configurable settings
 */
class RateLimiter {
  constructor(options = {}) {
    this.maxRequests = options.maxRequests ||
        DEFAULT_MAX_REQUESTS;
    this.windowMs = options.windowMs || DEFAULT_WINDOW_MS;
    this.storage = chrome.storage.local;
  }

  /**
   * Check if a request is allowed
   * @param {string} action - Action being rate limited
   * @returns {Promise<{allowed: boolean, remaining: number,
       resetTime: number}>}
   */
  async isAllowed(action) {
    const key = `${RATE_LIMIT_PREFIX}${action}`;
    const now = Date.now();
    const windowStart = now - this.windowMs;

    const result = await this.storage.get(key);
    let timestamps = result[key] || [];

    // Remove timestamps outside the current window
    timestamps = timestamps.filter(ts => ts > windowStart);

    if (timestamps.length >= this.maxRequests) {
      const resetTime = timestamps[0] + this.windowMs;
      return {
        allowed: false,
        remaining: 0,
        resetTime
      };
    }

    // Add current request timestamp
    timestamps.push(now);
    await this.storage.set({ [key]: timestamps });

    return {
```

```
      allowed: true,
      remaining: this.maxRequests - timestamps.length,
      resetTime: now + this.windowMs
    };
  }

  /**
   * Reset rate limit for an action
   * @param {string} action
   */
  async reset(action) {
    const key = `${RATE_LIMIT_PREFIX}${action}`;
    await this.storage.remove(key);
  }
}

// Global rate limiter instance
let rateLimiter = new RateLimiter();

/**
 * Update rate limiter configuration from user settings
 */
async function updateRateLimiterConfig() {
  const result = await chrome.storage.local.get('boba_settings');
  const settings = result.boba_settings || {};

  rateLimiter = new RateLimiter({
    maxRequests: settings.maxRequestsPerMinute ||
        DEFAULT_MAX_REQUESTS,
    windowMs: settings.rateLimitWindowMs || DEFAULT_WINDOW_MS
  });
}

/**
 * Middleware: Check rate limit before executing API action
 */
async function withRateLimit(action, asyncFn) {
  const check = await rateLimiter.isAllowed(action);

  if (!check.allowed) {
    const waitSeconds = Math.ceil((check.resetTime -
        Date.now()) / 1000);
    throw new RateLimitError(
      `Rate limit exceeded for ${action}. Try again in $
        {waitSeconds}s.`
    );
  }

  return asyncFn();
}

class RateLimitError extends Error {
```

```
  constructor(message) {
    super(message);
    this.name = 'RateLimitError';
    this.retryable = true;
  }
}

// Background message handler with rate limiting
chrome.runtime.onMessage.addListener((request, sender,
        sendResponse) => {
  // Validate sender
  if (sender.id !== chrome.runtime.id) return;
  if (!isValidAction(request.action)) return;

  (async () => {
    try {
      switch (request.action) {
        case 'ADD_MAGNET':
          const result = await withRateLimit('add_magnet', async
        () => {
            return await addMagnetToBoba(request.data);
          });
          sendResponse({ success: true, data: result });
          break;

        case 'GET_STATUS':
          const status = await withRateLimit('get_status', async
        () => {
            return await getBobaStatus();
          });
          sendResponse({ success: true, data: status });
          break;

        // ... other actions
      }
    } catch (error) {
      if (error instanceof RateLimitError) {
        sendResponse({ success: false, error: error.message,
        rateLimited: true });
      } else {
        sendResponse({ success: false, error: error.message });
      }
    }
  })();

  return true; // Keep channel open for async
});
```

11.2 Rate Limit Configuration

```
// User-configurable rate limits (stored in settings)
const RATE_LIMIT_PRESETS = {
```

```
  conservative: { maxRequests: 10, windowMs: 60000 },
  normal: { maxRequests: 30, windowMs: 60000 },
  relaxed: { maxRequests: 60, windowMs: 60000 },
  custom: null  // User-defined
};
```

---

12. Privacy Architecture

12.1 Data Collection Matrix

Data Category	Collected?	Purpose	Storage	Retention
Boba server URL	Yes (user-provided)	Connect to Boba	Encrypted local	Until deleted
Boba credentials	Yes (user-provided)	Authenticate with Boba	Encrypted local	Until deleted
Magnet links	Yes (extracted from pages)	Send to Boba	Not stored (ephemeral)	Immediate discard
Torrent names	Yes (from magnet links)	Display in UI	Not stored	Immediate discard
Browser tabs/URLs	Yes (activeTab only)	Detect magnet links	Not stored	Immediate discard
Page content	No	N/A	N/A	N/A
Browsing history	No	N/A	N/A	N/A
IP address	No (indirect via Boba)	N/A	N/A	N/A
Extension usage stats	Optional (opt-in)	Improve extension	Local only if enabled	30 days
Error logs	Optional (opt-in)	Debug issues	Local only	7 days

12.2 Privacy Principles

Claim: Google Chrome Web Store requires extensions to follow “Limited Use” policy — user data must only be used to provide or improve the extension’s single purpose. Data transfers are limited to specific purposes, and selling user data to data brokers or for advertising is prohibited. Source: Herzog Law / Google Chrome Web Store Policy URL: https://herzoglaw.co.il/en/news-and-insights/new-privacy-requirements-for-google-chrome-extensions/ Date: 2020-12-15 Excerpt: “The use of users’ data must be limited to providing or improving the extension’s single purpose. In addition, transfers of data are limited to certain purposes… Any of these actions is prohibited when the purpose is personalized advertisements or determining credit-worthiness.” Context: Chrome Web Store privacy requirements for extensions Confidence: high

12.3 Privacy Controls

```
// privacy-controls.js — User privacy settings

const DEFAULT_PRIVACY_SETTINGS = {
  // Data collection opt-outs
  enableAnalytics: false,        // Opt-in only
  enableErrorReporting: false,   // Opt-in only

  // Data retention
  autoClearHistory: true,        // Clear action history on
        browser close
  historyRetentionDays: 7,       // How long to keep local action
        log

  // Security settings
  requirePasswordOnStartup:
        true,  // Require master password when browser starts
  autoLockTimeoutMinutes: 30,      // Auto-lock after inactivity

  // Communication settings
  verifyCertificates: true,        // Validate Boba server
        certificates
  enforceHttps:
        true,              // Block non-HTTPS Boba servers (except
        localhost)

  // Content script scope
  enabledSites: [                  // Which sites to inject
        content script
    'thepiratebay',
    '1337x',
    'nyaa',
    'yts',
    'eztv'
```

```
  ]
};

/**
 * Get privacy settings (with defaults)
 */
async function getPrivacySettings() {
  const result = await
        chrome.storage.local.get('boba_privacy_settings');
  return
        { ...DEFAULT_PRIVACY_SETTINGS, ...result.boba_privacy_settings };
}

/**
 * Save privacy settings
 */
async function savePrivacySettings(settings) {
  await chrome.storage.local.set({
    boba_privacy_settings: settings
  });
}

/**
 * Clear all extension data (GDPR "right to erasure")
 */
async function eraseAllData() {
  // Clear all storage
  await chrome.storage.local.remove([
    'boba_credentials_encrypted',
    'boba_settings',
    'boba_privacy_settings',
    'boba_session_cache'
  ]);
  await chrome.storage.session.remove(['boba_session_cache']);

  // Notify user
  chrome.notifications.create('data-erased', {
    type: 'basic',
    iconUrl: 'icons/icon48.png',
    title: 'Boba: Data Erased',
    message: 'All stored data has been permanently deleted.'
  });
}
```

12.4 Third-Party Data Sharing

Third Party	Data Shared	Purpose	User Consent
Self-hosted Boba	Magnet links, torrent URLs	Adding torrents to download	Required (core function)

Third Party	Data Shared	Purpose	User Consent
Chrome Web Store	Extension version, install count	Distribution	Implicit (Store policy)
No analytics services	—	—	—
No advertising networks	—	—	—
No data brokers	—	—	—

13. Secure Communication

13.1 TLS/SSL Requirements

Requirement	Level	Implementation
TLS Version	Minimum 1.2	Server configuration
Certificate	Valid and trusted	System CA store or user-installed
HSTS	Recommended	Server response header
Cipher Suites	Modern only	No deprecated ciphers
Certificate Transparency	Recommended	For public-facing Boba

13.2 Custom CA Handling

For self-hosted Boba servers using a private CA or self-signed certificate:

```
User Workflow:
1. User generates self-signed cert for Boba server
2. User installs cert in OS trust store
3. Extension validates cert via standard HTTPS
4. Extension warns if cert is not trusted
5. Optional: User can pin certificate fingerprint
```

Claim: For Chrome to accept a self-signed certificate, the user must import it into the “Trusted Root Certification Authorities” store. Chrome respects the system certificate store. Source: Pico.net URL: https://www.pico.net/kb/how-do-you-get-chrome-to-accept-a-self-signed-certificate/ Date: N/A Excerpt: “Navigate to the site with the cert you want to trust… open the Chrome settings page, scroll to the bottom… in the ‘Privacy and security’

panel, click on the ‘Manage certificates’ area… select the ‘Trusted Root Certification Authorities’ tab, and click on the ‘Import…’ button” Context: Procedure for trusting self-signed certificates in Chrome Confidence: high

13.3 Fetch Security Configuration

```
/**
 * Secure fetch wrapper with all security options
 */
async function secureFetch(url, options = {}) {
  return fetch(url, {
    ...options,
    // Security options
    credentials: 'omit',           // Never send browser cookies
    referrerPolicy: 'no-referrer', // Don't leak referrer URL
    cache: 'no-store',             // Don't cache sensitive data
    redirect: 'error',             // Don't follow redirects
        (prevent open redirect)
    integrity: options.integrity || undefined,  // Subresource
        Integrity if available

    headers: {
      'X-Requested-With': 'XMLHttpRequest',  // CSRF protection
        signal
      ...options.headers
    }
  });
}
```

---

14. Extension Update Security

14.1 Update Mechanism

Claim: Browser extensions are automatically updated through the browser’s extension store (Chrome Web Store, Firefox Add-ons). The update manifest must be hosted on an HTTPS server for Firefox extensions. Updates are cryptographically signed by the store. Source: Firefox Extension Workshop URL: https://extensionworkshop.com/documentation/manage/updating-your-extension/ Date: 2026-03-22 Excerpt: “Firefox supports automated updates to add-ons using JSON update manifests. Add-ons hosted on addons.mozilla.org (AMO) automatically receive updates to new versions posted there. Other add-ons must specify the location of their update manifests. You must host your update manifest file on a secure (HTTPS) server.” Context: Extension update requirements and security Confidence: high

14.2 Update Security Properties

Property	Chrome	Firefox	Edge
Auto-update	Yes (Web Store)	Yes (AMO or self-hosted)	Yes (Edge Add-ons)
Signature verification	Yes (CRX signature)	Yes (XPI signature)	Yes
HTTPS required for manifest	N/A (store-managed)	Yes	N/A (store-managed)
Hash verification	No (implicit via signature)	Yes (update_hash)	No
User notification	Silent background	Silent background	Silent background

14.3 Update Manifest (Firefox Self-Hosted)

```
{
  "addons": {
    "boba-extension@example.com": {
      "updates": [
        {
          "version": "1.0.1",
          "update_link": "https://boba.example.com/releases/
        boba-1.0.1.xpi",
          "update_hash":
        "sha256:fe93c2156f05f20621df1723b0f39c8ab28cdbeec342efa95535d3abff932096"
        }
      ]
    }
  }
}
```

14.4 Preventing Malicious Updates

```
// Security: Monitor for unexpected permission changes on update
chrome.runtime.onInstalled.addListener((details) => {
  if (details.reason === 'chrome_update' || details.reason ===
        'browser_update') {
    // Extension was updated — verify no unexpected permission
        changes
    checkPermissions();
  }
});

async function checkPermissions() {
  const manifest = chrome.runtime.getManifest();
  const currentPermissions = new Set([
    ...manifest.permissions,
    ...manifest.host_permissions
```

```
  ]);

  // Log current permissions for security audit
  console.log('Extension permissions after update:',
        [...currentPermissions]);

  // Alert user if new permissions were added
  const previousPermissions = await
        chrome.storage.local.get('previous_permissions');
  if (previousPermissions.previous_permissions) {
    const newPerms = [...currentPermissions].filter(
      p => !previousPermissions.previous_permissions.includes(p)
    );

    if (newPerms.length > 0) {
      chrome.notifications.create('new-perms', {
        type: 'basic',
        iconUrl: 'icons/icon48.png',
        title: 'Boba: Permissions Updated',
        message: `New permissions added: ${newPerms.join(', ')}.
        Review in extension settings.`,
        requireInteraction: true
      });
    }
  }

  // Store current permissions for next comparison
  await chrome.storage.local.set({
    previous_permissions: [...currentPermissions]
  });
}
```

---

15. Sandboxed iframes

15.1 When Sandboxing is Needed

Sandboxed iframes should be used if the extension needs to: - Display external web content (e.g., torrent site previews) - Execute user-provided HTML/CSS - Run third-party JavaScript in isolation

15.2 Sandboxed Page Configuration

```
{
  "sandbox": {
    "pages": ["sandboxed.html"]
  },
  "content_security_policy": {
```

```
    "sandbox": "sandbox allow-scripts allow-forms allow-popups
        allow-modals; script-src 'self' 'unsafe-inline' 'unsafe-
        eval'; child-src 'self';"
  }
}
```

15.3 Sandboxed iframe Security Pattern

```
// sandbox-bridge.js — Secure communication with sandboxed iframe

// In extension page (popup/background):
const sandbox = document.createElement('iframe');
sandbox.src = chrome.runtime.getURL('sandboxed.html');
sandbox.sandbox = 'allow-scripts';  // Minimal sandbox permissions
sandbox.setAttribute('csp', "default-src 'none'; script-src
        'self';");

document.body.appendChild(sandbox);

// Message passing with sandbox (validate origin)
window.addEventListener('message', (event) => {
  // CRITICAL: Validate origin is our sandbox
  if (event.origin !== chrome.runtime.getURL('').replace(/\/$/,
        '')) {
    return;
  }

  // Handle sandbox messages
  if (event.data.type === 'SANDBOX_RESULT') {
    processSandboxResult(event.data.payload);
  }
});
```

15.4 Sandboxing Recommendations for Boba Extension

Feature	Sandboxing Needed?	Recommendation
Magnet link display	No	Native DOM manipulation
Torrent status from Boba	No	Direct API calls
Options/ settings page	No	Extension page with CSP
External torrent site preview	**Yes**	Use sandboxed iframe with no privileges
	Yes	

Feature	Sandboxing Needed?	Recommendation
User-provided custom CSS		Sandbox with `allow-scripts` only

16. Security Best Practices Checklist

Manifest & Permissions

- [ ] Use minimum required permissions (storage, alarms, notifications, activeTab)
- [ ] Declare `host_permissions` separately from `permissions`
- [ ] Use `optional_permissions` for non-essential features
- [ ] Never request `<all_urls>`, `tabs`, `cookies`, `history`, `downloads`
- [ ] Set `run_at: "document_idle"` for content scripts
- [ ] Set `all_frames: false` unless absolutely required
- [ ] Use `use_dynamic_url: true` in `web_accessible_resources`
- [ ] Restrict `web_accessible_resources` to specific `matches` patterns
- [ ] Never include HTML pages in `web_accessible_resources`
- [ ] Define strict CSP in `content_security_policy.extension_pages`

Storage & Credentials

- [ ] Encrypt all credentials with AES-256-GCM before storing
- [ ] Use PBKDF2 with at least 100,000 iterations for key derivation
- [ ] Never store master password (obtain from user per session)
- [ ] Use `chrome.storage.session` for decrypted credential cache
- [ ] Clear session storage on browser restart

- [ ] Provide “Clear All Data” function for GDPR compliance

Communication

- [ ] Validate `sender.id` on ALL `chrome.runtime.onMessage` handlers
- [ ] Validate `sender.url` starts with `chrome-extension://`
- [ ] Allow-list all action names; reject unknown actions
- [ ] Use HTTPS for all external communication
- [ ] Block non-HTTPS Boba servers (except localhost)
- [ ] Set `credentials: 'omit'` on all fetch calls
- [ ] Set `referrerPolicy: 'no-referrer'` on all fetch calls
- [ ] Validate all responses before processing

Input Validation

- [ ] Validate all magnet links with regex before processing
- [ ] Validate info-hash length (40 hex chars for v1, 64 for v2)
- [ ] Sanitize all display names before DOM insertion
- [ ] Use `textContent` not `innerHTML` for user data
- [ ] Validate Boba URL format (protocol, hostname, no credentials)
- [ ] Reject URLs with `javascript:`, `data:`, `vbscript:` protocols
- [ ] Block dangerous ports in URLs

Content Scripts

- [ ] Use specific `matches` patterns (never `<all_urls>`)
- [ ] Use `JSON.parse` not `eval` for data parsing
- [ ]

- [ ] Never execute dynamically constructed code
- [ ] Validate `event.origin` in all `postMessage` handlers
- [ ] Treat all page DOM data as untrusted
- [ ] Keep content script surface minimal

Rate Limiting

- [ ] Implement client-side rate limiting for Boba API
- [ ] Use `chrome.alarms` for rate limit window management
- [ ] Make rate limits user-configurable
- [ ] Provide clear error messages when rate limited
- [ ] Log rate limit events for debugging

Update Security

- [ ] Distribute only through official stores
- [ ] Verify permissions haven’t changed on update
- [ ] Notify users of new permissions
- [ ] Use HTTPS for self-hosted update manifests
- [ ] Include `update_hash` for Firefox self-hosted updates

General

- [ ] No `eval()`, `new Function()`, or inline scripts
- [ ] No `innerHTML` with untrusted data
- [ ] No hardcoded API keys, credentials, or URLs
- [ ] No remote script loading
- [ ] No analytics or tracking without explicit opt-in

- [ ] No data selling or sharing with third parties
- [ ] Provide clear privacy policy

---

17. Privacy Policy Template

```
# Privacy Policy — Boba Torrent Manager Extension

**Last Updated:** [Date]
**Effective Date:** [Date]

## Overview

Boba Torrent Manager is a browser extension that allows you to
        send torrent magnet
links and .torrent files to your self-hosted Boba server. This
        privacy policy
describes what data we collect, how we use it, and your rights.

## Data We Collect

### User-Provided Data
- **Boba Server URL**: The URL of your self-hosted Boba server
        (stored encrypted locally)
-
        **Boba Credentials**: Username/password or API key for
        your Boba server (stored encrypted locally)
- **Extension Settings**: User preferences (stored locally)

### Automatically Processed Data
- **Magnet Links**: Extracted from torrent sites you visit. These
        are sent directly
  to your Boba server and are **never stored** by the extension.
- **Active Tab URL**: Used only to detect magnet links on the
        current page.
  **Not stored or transmitted**.

### Data We Do NOT Collect
- Browsing history
- Page content (other than magnet links)
- IP addresses
- Personal identifiers
- Cookies from websites
- Download history

## How We Use Data

| Data | Purpose | Legal Basis |
|------|---------|-------------|
```

Boba Server URL	Connect to your self-hosted server	Performance of contract (your configuration)
Boba Credentials	Authenticate with your server	Performance of contract
Magnet Links	Send torrents to your server	Your explicit action
Extension Settings	Provide configured features	Legitimate interest

Data Storage and Security

- All sensitive data is **encrypted using AES-256-GCM** before storage
- Encryption keys are derived from your master password using PBKDF2
- Your master password is **never stored** — you provide it each session
- Decrypted credentials may be cached in **memory only** (lost on browser restart)
- No data is sent to any server other than your configured Boba server

Data Sharing

We do **not** share, sell, or transfer your data to any third parties.

Data is transmitted only to:
- Your self-hosted Boba server (as configured by you)
- The browser extension store (for updates only)

Data Retention

Data Type	Retention Period
Encrypted credentials	Until you delete them or uninstall the extension
Session cache	Until browser is closed
Settings	Until you delete them or uninstall the extension
Error logs (if enabled)	7 days

Your Rights

Under GDPR and applicable privacy laws, you have the right to:

1. **Access**: Request a copy of your stored data
2. **Rectification**: Update incorrect data
3. **Erasure**: Delete all your data (use the "Clear All Data" button in settings)
4. **Restriction**: Limit processing of your data
5. **Portability**: Export your settings
6. **Objection**: Object to data processing

```
To exercise these rights, use the extension's settings page or
        contact us.

## Opt-Out Options

- **Analytics**: Disabled by default. Enable in settings only if
        you wish to help improve the extension.
- **Error Reporting**: Disabled by default. Enable in settings to
        help us fix bugs.
- **Content Script Sites**: Choose which torrent sites the
        extension interacts with.

## Changes to This Policy

We will notify you of significant changes through:
- The extension's update notes
- A notification in the extension popup

## Contact

For privacy questions or to exercise your rights, contact:
[Your Contact Information]

## Compliance

This extension complies with:
- Google Chrome Web Store Developer Program Policies
- Mozilla Add-on Policies
- General Data Protection Regulation (GDPR)
- California Consumer Privacy Act (CCPA)
```

---

18. References

Official Documentation

1. **Chrome Extension Manifest V3**: https://developer.chrome.com/docs/extensions/mv3/intro/
2. **Content Security Policy (MV3)**: https://developer.chrome.com/docs/extensions/reference/manifest/content-security-policy
3. **chrome.storage API**: https://developer.chrome.com/docs/extensions/reference/api/storage
4. **chrome.alarms API**: https://developer.chrome.com/docs/extensions/reference/api/alarms
5. **Content Scripts**: https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts
6. **MV3 Security Improvements**: https://developer.chrome.com/docs/extensions/develop/migrate/improve-security

7. **Firefox Extension Security**: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Security_best_practices
8. **Firefox Update Manifest**: https://extensionworkshop.com/documentation/manage/updating-your-extension/
9. **Chrome Web Store User Data Policy**: https://developer.chrome.com/docs/webstore/program-policies/user-data-faq
10. **OWASP Browser Extension Cheat Sheet**: https://cheatsheetseries.owasp.org/cheatsheets/Browser_Extension_Vulnerabilities_Cheat_Sheet.html

Security Research

1. **Universal Code Execution in Browser Extensions** (Space Raccoon): https://spaceraccoon.dev/universal-code-execution-browser-extensions/
2. **Chrome Extensions: Threat Analysis and Countermeasures** (NDSS Symposium): https://www.ndss-symposium.org/wp-content/uploads/2017/09/P11_4.pdf
3. **Attacking Browser Extensions** (GitHub Security Research): https://github.blog/security/vulnerability-research/attacking-browser-extensions/
4. **When Extension Pages are Web-Accessible** (Palant.info): https://palant.info/2022/08/31/when-extension-pages-are-web-accessible/
5. **Web Crypto API Security**: https://blog.doubleslash.de/en/software-technologien/cyber-security/web-crypto-api-security-and-encryption-on-the-web

Standards

1. **Magnet URI Scheme (Wikipedia)**: https://en.wikipedia.org/wiki/Magnet_URI_scheme
2. **BitTorrent BEP 9**: http://bittorrent.org/beps/bep_0009.html
3. **STRIDE Threat Model (Wikipedia)**: https://en.wikipedia.org/wiki/STRIDE_model
4. **OWASP Threat Modeling**: https://owasp.org/www-community/Threat_Modeling_Process
5. **Web Crypto API (MDN)**: https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API

Document generated for Boba Torrent Browser Extension security architecture review. This document should be reviewed quarterly and updated as threats evolve.